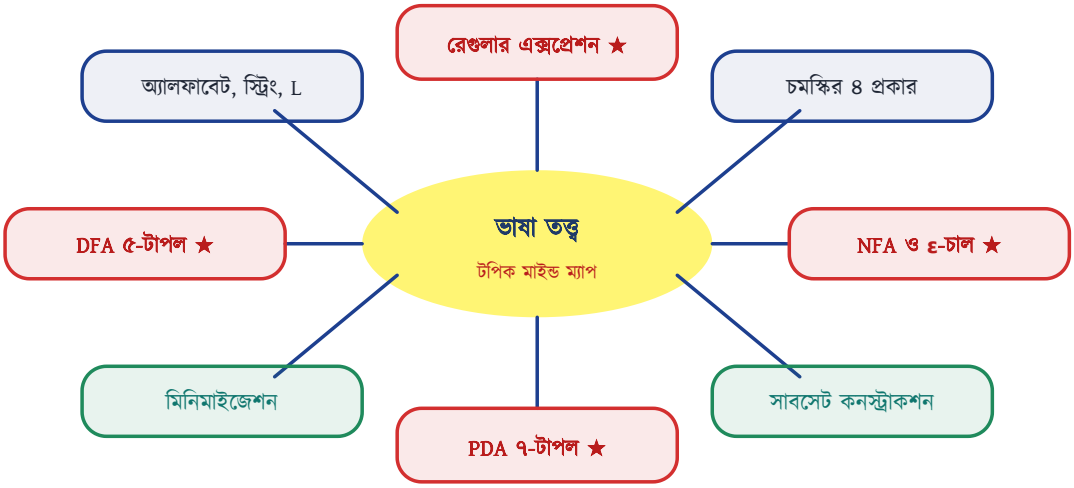
# ভাষা তত্ত্ব ও ফাইনিট অটোমাটা

**অ্যালফাবেট ও ভাষা · চমস্কির প্রকার · DFA · NFA · DFA = NFA · PDA: ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি**




↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = পদ্ধতি / প্রয়োগ

## সিলেবাসের যে অংশ

"Language theory; Finite automata: **deterministic finite automata**, **nondeterministic finite automata**, **equivalence and conversion** of deterministic and nondeterministic finite automata, **pushdown automata**; Context free languages; Context free grammars; Turing Machines: basic machines, configuration, computing with Turing machines." এই নোটে আছে ভাষার মূল ধারণা, ফাইনিট অটোমাটা ও পাশডাউন অটোমাটা; গ্রামারের বিস্তারিত CFL ও CFG নোটে, টিউরিং মেশিন পরের নোটে।

**হাই-ইল্ড তালিকা**

① DFA ৫-টাপল, PDA ৭-টাপল
② সম্পূর্ণ DFA: ঠিক ১টি চাল
③ ক্ষমতায় DFA = NFA
④ সাবসেট → সর্বোচ্চ $2^n$ অবস্থা
⑤ PDA = কনটেক্সট-ফ্রি
⑥ পাম্পিং লেমা: রেগুলার নয়

**সংকেত:** **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = মুখস্থ রাখার তথ্য · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

## সূচিপত্র


✓ অ্যালফাবেট, স্ট্রিং, ভাষা ..... পৃ. ২
✓ চমস্কির চার প্রকার ..... পৃ. ৩
✓ FA ৫-টাপল ও DFA ..... পৃ. ৪
✓ NFA ও ε-চাল ..... পৃ. ৫
✓ DFA = NFA, সাবসেট কনস্ট্রাকশন ..... পৃ. ৬
✓ মিনিমাইজেশন, পাম্পিং লেমা ..... পৃ. ৭
✓ পাশডাউন অটোমাটন ..... পৃ. ৮
✓ DFA বনাম NFA বনাম PDA ..... পৃ. ৯
✓ পরীক্ষার আগের রাতের পাতা ..... পৃ. ১০
✓ অনুশীলন MCQ + লিখিত ..... পৃ. ১১

# ১ অ্যালফাবেট, স্ট্রিং ও ভাষা

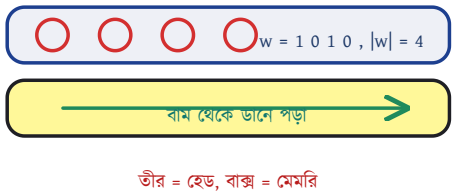
**সংজ্ঞা:** **ভাষা তত্ত্ব** (language theory) আলোচনা করে **ফর্মাল ভাষা**: নির্দিষ্ট সসীম অ্যালফাবেটের উপর গড়ে ওঠা স্ট্রিংয়ের সেট। কোনো মেশিন একটি ভাষাকে **গ্রহণ** (accept) করে বলে যদি সে ঠিক **ওই** ভাষার স্ট্রিংগুলোর জন্যই "হ্যাঁ" বলে, বাকি সবের জন্য "না"। এই অধ্যায়ের বাকি সবই এমন একটি সেট ঠিক করার মেশিন।

| পরিভাষা | অর্থ | উদাহরণ, $\Sigma = \{0, 1\}$ |
|---|---|---|
| অ্যালফাবেট $\Sigma$ | প্রতীকের সসীম, খালি-নয় সেট | $\Sigma = \{0, 1\}$ |
| স্ট্রিং $w$ | প্রতীকের সসীম ধারাবাহিকতা; $\|w\|$ = দৈর্ঘ্য | $w = 1010$, $\|w\| = 4$ |
| $\varepsilon$ | **খালি স্ট্রিং**, $\|\varepsilon\| = 0$, $\varepsilon \in \Sigma^*$ | কোনো প্রতীকই নেই |
| $\Sigma^*$ | $\Sigma$-এর সব স্ট্রিং, $\varepsilon$-সহ | $\{\varepsilon, 0, 1, 00, 01, 10, 11, \ldots\}$ |
| ভাষা $L$ | $\Sigma^*$-এর যেকোনো উপসেট | $L = \{w \mid w$ 0-এ শেষ হয়$\}$ |
| $\Sigma^n$ | ঠিক $n$ দৈর্ঘ্যের স্ট্রিং; $\|\Sigma^n\| = \|\Sigma\|^n$ | $\Sigma^2 = \{00, 01, 10, 11\}$ |

## রেগুলার এক্সপ্রেশন: চারটি অপারেটর

| অপারেটর | অর্থ | কীভাবে পড়া যায় |
|---|---|---|
| a + b | ইউনিয়ন: a **অথবা** b | "প্লাস মানে বা" |
| ab | কনক্যাটেনেশন: a তারপর b | "পাশাপাশি লেখা" |
| (a + b)* | ক্লিন স্টার: **শূন্য** বা তার বেশি | "যত খুশি" |
| (a + b)+ | এক বা তার বেশি | "অন্তত একবার" |

```
(a+b)*  = a*b* = (a+b)+ U {e}
(a+b)+  = Sigma* \ {e}
ab*a    = {a, aa, aba, abba, ...}
```




স্ট্রিং মানে একটি হেডের নিচে সাজানো প্রতীক

**মনে রাখার কৌশল:** **স্টার = শূন্য বা তার বেশি, প্লাস = এক বা তার বেশি।** তাই $(a+b)^*$-এ **$\varepsilon$ আছে**, $(a+b)+$-এ নেই।

**পরীক্ষার ফাঁদ:** $ab^*a$ মানে "একটি a, তারপর **শূন্য** বা তার বেশি b, তারপর আরেকটি a" — তাই b শূন্য হলে স্ট্রিংটি **aa**। আর "w-তে **01 আছে**" ≠ "w **01-এ শেষ** হয়": ফাইনিট অটোমাটান শেষের অংশ মনে রাখতে পারে, কিন্তু মাঝের যেকোনো ঘটনা মনে রাখতে পারে না।

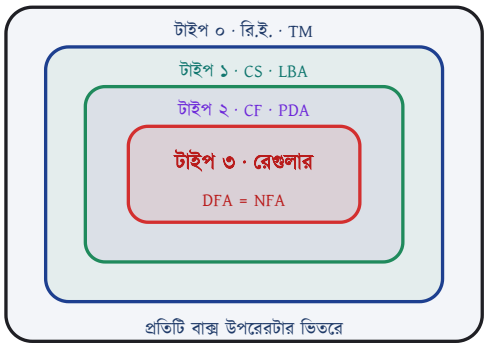
**ভাষার উপর ক্রিয়া:** ভাষা আসলে সেট, তাই **সেট** ক্রিয়া নেওয়া যায়: $L_1 \cup L_2$, $L_1 \cap L_2$, $L^c = \Sigma^* - L$ (সবসময় একই $\Sigma$-এর উপর), $L_1 \cdot L_2 = \{xy \mid x \in L_1, y \in L_2\}$ এবং $L^*$ ($\varepsilon$-সহ)। পূরক নেওয়া হয় **$\Sigma^*$**-এর উপর, "মেশিনের উপর" নয়: $\Sigma = \{0,1\}$ হলে "0-এ শেষ হয়"-এর পূরক হলো "1-এ শেষ হয় **অথবা** খালি স্ট্রিং"।

> $L$ হলো $\Sigma^*$-এর যেকোনো উপসেট — $\emptyset$ বা পুরো $\Sigma^*$ ও হতে পারে

## ২ চমস্কি শ্রেণিবিন্যাস: চার প্রকার

**সংজ্ঞা:** চমস্কির (Chomsky) ফর্মাল ভাষার শ্রেণিবিন্যাস — **চারটি পরস্পরের ভিতরে বাঁধা** শ্রেণি, প্রতিটি গ্রামারের একটি ধরন দিয়ে সংজ্ঞায়িত এবং একটি মেশিনের সঙ্গে মিলিয়ে দেওয়া। টাইপ **n** টাইপ **n−1**-এর ভিতরে: প্রতিটি রেগুলার ভাষাই সেই সঙ্গে কনটেক্সট-ফ্রি, এভাবে উপরের দিকে সব ধরে।

| প্রকার | গ্রামার | অটোমাটন | উদাহরণ ভাষা |
|---|---|---|---|
| **টাইপ ০** রিকার্সিভলি এনিউমারেবল | আনরেস্ট্রিক্টেড | টিউরিং মেশিন | {w # w \| w ∈ {0,1}*} |
| **টাইপ ১** কনটেক্সট-সেনসিটিভ | লিনিয়ার বাউন্ডেড (LBA) | LBA | {$a^n b^n c^n$ \| n ≥ 1} |
| **টাইপ ২** কনটেক্সট-ফ্রি | কনটেক্সট-ফ্রি (CFG) | **পাশডাউন অটোমাটন** | {$a^n b^n$ \| n ≥ 0} |
| **টাইপ ৩** রেগুলার | রেগুলার | **ফাইনিট অটোমাটন** | {w \| w 01-এ শেষ হয়} |




কঠোর অন্তর্ভুক্তি: 0 ⊃ 1 ⊃ 2 ⊃ 3

**মেশিন ↔ ভাষা, এক লাইনের উত্তর:**
FA (DFA / NFA) ⇒ **রেগুলার** = টাইপ ৩ · PDA ⇒ **কনটেক্সট-ফ্রি** = টাইপ ২
LBA ⇒ **কনটেক্সট-সেনসিটিভ** = টাইপ ১ · TM ⇒ **রিকার্সিভলি এনিউমারেবল** = টাইপ ০

**মনে রাখার কৌশল:** ছকটি **উপরের দিকে** পড়ো: প্রতিধাপে এক ধরনের মেমরি যোগ হয় — **কিছুই না** (FA) → **স্ট্যাক** (PDA) → **টেপ** (LBA, TM)। বেশি মেমরি ⇒ বেশি ভাষা।

**পরীক্ষার ফাঁদ:** {$a^n b^n c^n$}-এর জন্য PDA নয়, **লিনিয়ার বাউন্ডেড অটোমাটন** লাগে: একটি স্ট্যাক অন্তত **একটি** ভারসাম্য গণনা করতে পারে, তাই $a^n b^n$ নিলে কাজ হয়, কিন্তু $a^n b^n c^n$ কখনো নয়। টাইপ ০-এর উদাহরণে মাঝ খুঁজতে "#" দরকার।

**পাশের নোট:** কনটেক্সট-ফ্রি **গ্রামার** (প্রোডাকশন, ডেরিভেশন ট্রি, পার্সিং) CFL ও CFG নোটে; টিউরিং মেশিন, কনফিগারেশন ও কমপিউটেবিলিটি টিউরিং মেশিন নোটে। দুটোই এই অধ্যায়ের অটোমাটন যে টাইপ ২ ও টাইপ ১ ভাষার নাম ধরে, সেগুলোই বোঝায়।

**IT link:** *grep*, *RegExp*, *lex*-এর ভিতরের রেগুলার ইঞ্জিন আসলে ফাইনিট অটোমাটন: কোনো ভাষার **সিনট্যাক্স** একটি রেগুলার (টাইপ ৩) ভাষা, আর তার **গ্রামার** কনটেক্সট-ফ্রি (টাইপ ২) — তাই পার্সিংয়ে স্ট্যাক লাগেই।

## ৩ ফাইনিট অটোমাটন: ৫-টাপল

**সংজ্ঞা:** **ফাইনিট অটোমাটন** M = (Q, Σ, δ, $q_0$, F) স্ট্রিং বাম থেকে ডানে পড়ে, প্রতি প্রতীকে অবস্থা বদলে শেষে **গ্রহণ** বা **প্রত্যাখ্যান** করে। বর্তমান অবস্থার বাইরে কোনো **মেমরি** নেই, তাই এটি কেবল রেগুলার ভাষাই চিনতে পারে।

| অংশ | কী | অর্থ |
|---|---|---|
| Q | অবস্থার সসীম সেট, $\|Q\| = n$ | ডায়াগ্রামের বাক্সগুলো; $q_0 \in Q$ |
| **Σ** | ইনপুট অ্যালফাবেট (সসীম, খালি-নয়) | টেপে যে প্রতীক থাকতে পারে |
| **δ** | ট্রানজিশন ফাংশন, $\delta: Q \times \Sigma \to Q$ (DFA) | তীরগুলো: প্রতীক পড়ে অবস্থা বদলাও |
| $q_0$ | একটিমাত্র **শুরু** অবস্থা | হেড যেখান থেকে শুরু করে |
| F | **গ্রহণকারী/চূড়ান্ত** অবস্থার সেট, $F \subseteq Q$ | চলা F-এ শেষ হলেই গ্রহণ |

**মনে রাখার কৌশল:** পাঁচটি অংশের ক্রম: **Q**uestions, **Σ**ymbols, **δ**ecision rule, quest**0** entry, **F**inish states। PDA-তে যোগ হয় **Γ** ও $Z_0$ = **৭**।

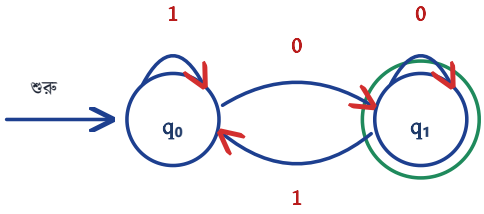
## ৪ ডিটারমিনিস্টিক ফাইনিট অটোমাটন (DFA)

**সংজ্ঞা:** **প্রতিটি** (অবস্থা, প্রতীক) জোড়ার জন্য ঠিক **একটি** পরবর্তী অবস্থা আছে: $\delta: Q \times \Sigma \to Q$ একটি **টোটাল** ফাংশন। কোনো ε-চাল নেই, কোনো অনুমান নেই।

### L = {w | w 0-এ শেষ হয়}

| অবস্থা | 0 পড়লে | 1 পড়লে | অর্থ |
|---|---|---|---|
| → $q_0$ | $q_1$ | $q_0$ | শেষ প্রতীক ছিল 1 |
| * $q_1$ | $q_1$ | $q_0$ | শেষ প্রতীক ছিল 0 |

$\Sigma = \{0,1\}$ · → শুরু · * গ্রহণকারী। **1100:** $q_0$ -1→ $q_0$ -1→ $q_0$ -0→ $q_1$ -0→ $q_1 \in F \Rightarrow$ **গ্রহণ**। **1111:** $q_0$-তে শেষ, $q_0 \notin F \Rightarrow$ প্রত্যাখ্যান।



প্রতি অবস্থা থেকে 0 ও 1-এ একটি চাল

"w 0-এ শেষ হয়" — $q_1$ ডাবল বৃত্ত = গ্রহণকারী

**গ্রহণ ও প্রত্যাখ্যান, ফর্মালভাবে:** $\delta^*(q_0, w)$ = পুরো w পড়ার পরের অবস্থা, তাই $w \in L \Leftrightarrow \delta^*(q_0, w) \in F$; চাল না থাকলে প্রত্যাখ্যান — তাই ফাঁকা ঘরে ট্র্যাপ অবস্থা।

**পরীক্ষার ফাঁদ:** DFA-তে প্রতি (অবস্থা, প্রতীক)-এ ঠিক **একটি** ট্রানজিশন; "শূন্য বা বেশি" ও ε-চাল **NFA**-র বৈশিষ্ট্য। ফাঁকা ঘর মানে **অসম্পূর্ণ** মেশিন।

**মনে রাখার কৌশল:** ইনপুট **1** দিয়ে $q_0$ ও $q_1$ আলাদা হয়, তাই এই ২-অবস্থার DFA আগেই **মিনিমাল**।

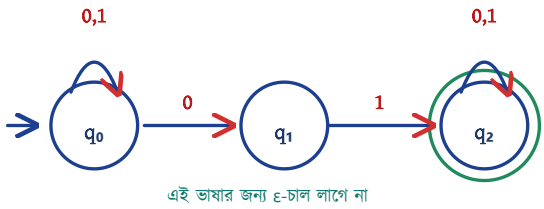
## ৫ ননডিটারমিনিস্টিক ফাইনিট অটোমাটন (NFA)

**সংজ্ঞা:** কোনো (অবস্থা, প্রতীক) জোড়ার জন্য NFA-তে **শূন্য, এক বা একাধিক** পরবর্তী অবস্থা থাকতে পারে — $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \to P(Q)$ — এবং এটি **ε-চাল** নিতে পারে: **কোনো ইনপুট না খেয়েই** অবস্থা বদলে ফেলতে পারে। ইনপুট শেষ হলে যদি **কোনো একটি** পথ গ্রহণকারী অবস্থায় পৌঁছায়, তবেই গ্রহণ করে।

### L = {w | w-তে 01 আছে}

| যেখান থেকে | 0 পড়লে | 1 পড়লে |
|---|---|---|
| → $q_0$ | $q_0$ এবং $q_1$ | $q_0$ |
| $q_1$ | — | $q_2$ |
| * $q_2$ | $q_2$ | $q_2$ |

$q_0$ অনুমান করে "এইমাত্র একটি 0 এসেছে"; পরের 1 তা নিশ্চিত করে, আর $q_2$ সেই মনে রাখা স্থায়ী করে। **0110:** $q_0$ -0→ {$q_0$,$q_1$} -1→ {$q_0$,$q_2$} -1→ {$q_0$,$q_2$} -0→ {$q_0$,$q_2$} ∋ $q_2$ ⇒ গ্রহণ। **110:** $q_0$ -1→ $q_0$ -1→ $q_0$ -0→ {$q_0$,$q_1$} ⇒ প্রত্যাখ্যান।



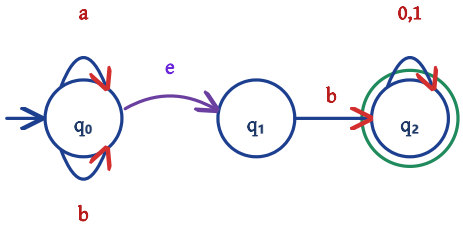
এই ভাষার জন্য ε-চাল লাগে না

"w-তে 01 আছে"-এর NFA: $q_2$ লুপ করে, তাই মনে রাখা টেকে

### ε-চাল সত্যিই কখন লাগে: L = {w | w-তে ab আছে}

মেশিনকে "ab"-এর অবস্থান ঠিক করতে হবে না, তাই এটি **একসঙ্গে দুইটি শাখা** চালায়: $q_0$ 0,1 লুপ করে স্ক্যান করতে থাকে, আর ε-চাল একটি কপি $q_1$-এ ফিরে যায়, যার একমাত্র কাজ হলো সেই b-কে দেখা, যা a নিশ্চিত করে। ε-চাল মানে NFA-র **ক্লোন** করার কৌশল — ইনপুট খরচ হয় না, শুধু অবস্থা বদলায়।

```
q0 --a--> q0      q0 --b--> q0
q0 --e--> q1      q1 --b--> q2*
q2 --0,1--> q2
```



ε-চাল (বেগুনি) মাথাকে সামনে কপি করে, খরচ নেই

**পরীক্ষার ফাঁদ:** "শূন্য বা তার বেশি ট্রানজিশন" ও "ε অনুমোদিত" — এই দুটোই **NFA**-র বৈশিষ্ট্য। ε-চাল ছাড়া আর প্রতি জোড়ায় একটি চালও যদি থাকে, তবে সেটা আসলে **DFA**-ই, শুধু লেখার ধরন আলাদা — পরের পাতার পুরো বিষয়টাই এটা।

**সমাধানকৃত ট্রেস ✍️ "aab"-তে ε-NFA:** শুরু {$q_0$} → a: $q_0$ -a-> $q_0$, সঙ্গে ε-অনুলিপি $q_1$ ⇒ {$q_0$, $q_1$} · a: $q_0$-এর কাছে a-চাল ⇒ {$q_0$, $q_1$} · b: $q_0$ -b-> $q_0$ ও $q_1$ -b-> $q_2$ ⇒ {$q_0$, $q_2$} ∋ $q_2$ ⇒ **গ্রহণ**। যেখানে চাল নেই সেই শাখা নিজেই মুছে যায় — এটিই "শূন্য ট্রানজিশন"।

**NFA আসলে চালাতে হয় কীভাবে:** একটি অবস্থা নয়, **অবস্থার সেট** বহন করো: {$q_0$} দিয়ে শুরু; প্রতি প্রতীক a-এর পর সেটটি বদলাও **প্রতিটি** সদস্যের উত্তরসমূহের ইউনিয়ন দিয়ে; শেষ সেটটি F-এর সঙ্গে মিললে গ্রহণ। উপরের ট্রেসে তাই {$q_0$,$q_1$}, তারপর {$q_0$,$q_2$} লেখা হয় — আর পরের পাতায় এই সেটগুলোই DFA-র অবস্থা হয়ে যায়।

## ৬ DFA ও NFA-এর সমতা

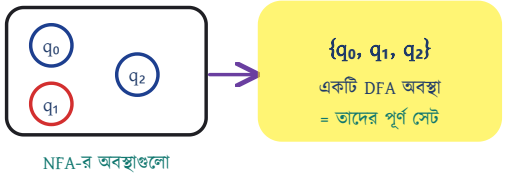
**উপপাদ্য (হাই-ইল্ড): DFA ও NFA-র এক্সপ্রেসিভ পাওয়ার হুবহু সমান:** দুটোই গ্রহণ করে ঠিক **রেগুলার** ভাষাগুলো। তাই **DFA → NFA তুলতামূলক** (NFA-তে প্রতি জোড়ায় একটি চাল থাকতে পারে, ফলে DFA আগেই একটি NFA), **NFA → DFA সবসময় সম্ভব**, NFA অনেক **সূচকীয়ভাবে ছোট** হতে পারে, আর DFA **দ্রুত** চলে কারণ পিছিয়ে অনুমান করার দরকার হয় না।

**প্রমাণের ছায়া (দুই দিক):** **DFA ⇒ NFA:** একই অবস্থা ও তীর রেখে দাও — NFA-র সংজ্ঞায় শূন্য বা একটি চালই অনুমোদিত। **NFA ⇒ DFA:** একটি DFA অবস্থা ধরে নাও NFA-র যে পূর্ণ **সেট** বোঝাবে, যেটা একই প্রিফিক্স পড়ার পর NFA যেসব জায়গায় থাকতে পারত; তাহলে w NFA-তে গৃহীত হবে ঠিক তখনই, যখন সেই সেটে একটি গ্রহণকারী অবস্থা আছে। ■

## সাবসেট (পাওয়ার-সেট) কনস্ট্রাকশন

১. একটি DFA অবস্থা = NFA অবস্থার একটি **সেট**; শুরু হয় ε-closure($q_0$) দিয়ে = $q_0$ ও ε-চালেই যেসব অবস্থায় পৌঁছানো যায়

২. **next = ε-closure(move(S, a))** — S-এর **প্রতিটি** সদস্যের উত্তরসমূহের ইউনিয়ন; সেট গ্রহণ করে যদি তার মধ্যে NFA-র কোনো গ্রহণকারী অবস্থা থাকে, আর খালি সেট ∅ হলো মৃত অবস্থা




প্রতিটি DFA অবস্থা আসলে NFA অবস্থার একটি বাক্স

**আকারের সীমা:** n-অবস্থার NFA থেকে সর্বোচ্চ **$২^n$**টি DFA অবস্থা (n টি অবস্থার $২^n$ উপসেট) — আর এই সীমা **নিবিদ্**: অনেক n-এর জন্য এমন ভাষা আছে যেগুলোতে প্রতিটি উপসেই আসে।

**পরীক্ষার ফাঁদ:** নিচের ছকে {$q_0$, $q_2$} ও {$q_0$, $q_1$, $q_2$}-এর চাল **একই**, তাই এরা জোড়া লাগে: মিনিমাল DFA-তে **৩**টি অবস্থা, ৪ নয়।

**সমাধানকৃত রূপান্তর ✍️:** ওপরের "w-তে 01 আছে" NFA-টি — প্রতিটি ঘর একটি **সেট**:

| DFA অবস্থা = একটি সেট | 0 পড়লে | 1 পড়লে | গ্রহণকারী? |
|---|---|---|---|
| → {$q_0$} | {$q_0$, $q_1$} | {$q_0$} | না |
| {$q_0$, $q_1$} | {$q_0$, $q_1$} | {$q_0$, $q_2$} | না |
| {$q_0$, $q_2$} | {$q_0$, $q_1$, $q_2$} | {$q_0$, $q_2$} | হ্যাঁ |
| {$q_0$, $q_1$, $q_2$} | {$q_0$, $q_1$, $q_2$} | {$q_0$, $q_2$} | হ্যাঁ |

**ট্রেস 0101:** {$q_0$} -0→ {$q_0$,$q_1$} -1→ {$q_0$,$q_2$} -0→ {$q_0$,$q_1$,$q_2$} -1→ {$q_0$,$q_2$} ⇒ গ্রহণ — মাত্র চারটি সেট, যেখানে ঊর্ধ্বসীমা $২^n$ = ১৬।

IT link: *grep -E* ও বেশিরভাগ রেগুলার ইঞ্জিন প্যাটার্নের জন্য একটি **NFA** বানায়, তারপর একসঙ্গে সব অবস্থা সিমুলেট করে (Thompson's construction), তারপর সাবসেট কনস্ট্রাকশন চালিয়ে DFA টেবিল বানায়: বিবরণ সস্তা, মেশিন দ্রুত।

**মনে রাখার কৌশল:** NFA = "হয়তো, একসঙ্গে"; DFA = "ঠিক এই একটি"। নির্দিষ্ট মেশিনের নাম বলতে বলা হলে টাপলের অংশ গোনো: **৫** = FA, **৭** = PDA।

# ৭ মিনিমাইজেশন ও পাম্পিং লেমা

## DFA মিনিমাইজ করা (টেবিল-ফিলিং)

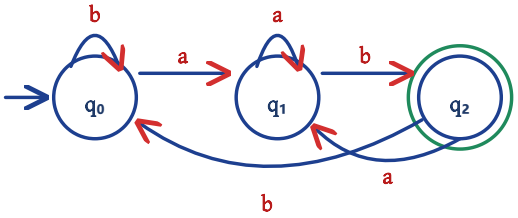
১. অপ্রাপ্য (unreachable) অবস্থা বাদ দাও — $q_0$ থেকে কোনো পথ নেই এমন

২. প্রতিটি (গ্রহণকারী, অগ্রহণকারী) জোড়া চিহ্নিত করো, তারপর আবার চালাও: কোনো প্রতীকে চিহ্নিত জোড়ায় পাঠালে (p, q)-কে চিহ্নিত করো

৩. চিহ্নিত নয় এমন জোড়া **পার্থক্যযোগ্য নয়** ⇒ একত্রীকরণ, তারপর টেবিল নতুন করে বানাও; উত্তর **নাম বদলানো ছাড়া একটাই**

## পাম্পিং লেমা (রেগুলার ভাষা)

L রেগুলার হলে এমন দৈর্ঘ্য p আছে যাতে $|w| \geq p$ ঐ প্রতিটি $w \in L$-কে $w = xyz$ করে ভাঙা যায়, যেখানে $|xy| < p$, $|y| > 0$ এবং **প্রতিটি $n \geq 0$-এর জন্য $xy^nz \in L$**। এর একমাত্র ব্যবহার হলো বিপরীত প্রমাপ: এমন একটি w খুঁজে বের করো যা পাম্প করা যায় না ⇒ **L রেগুলার নয়**।




"w ab-তে শেষ হয়" এর মিনিমাল DFA: $q_0$ কিছু মেলেনি, $q_1$ শেষে a ছিল, $q_2$ শেষের দুটি ab ছিল (গ্রহণকারী)

**পরীক্ষার ফাঁদ:** লেমা মেনে চললেই রেগুলার **প্রমাণ হয় না** — কিছু রেগুলার-বহির্ভূত ভাষাও লেমা মেনে চলে। "প্রমাণ করো এটি রেগুলার" — এই আকারের প্রশ্ন পাম্পিং লেমার নয়।

| ভাষা, {a,b}-এর উপর | রেগুলার? | মেশিন / অবস্থা |
|---|---|---|
| $a^* b^* \cdot (ab)^*$ | হ্যাঁ | DFA, 2–3 states |
| a-এর জোড় সংখ্যক | হ্যাঁ | DFA, 2 states (parity) |
| w ab-তে শেষ হয় | হ্যাঁ | DFA, 3 states |
| $\{a^n b^n \mid n \geq 0\}$ | **না** | PDA লাগবে |
| $\{w \mid \#a = \#b\}$ | **না** | PDA লাগবে |
| $\{a^n b^n c^n \mid n \geq 1\}$ | **না** | এমনকি CF-ও নয় — LBA / TM |

**সমাধানকৃত উদাহরণ ✍️ বাইনারি, 5-এর গুণিতীশ:**
বাইনারি সংখ্যা MSB আগে পড়া হবে; তার মান $\equiv 0 \pmod 5$ হলে গ্রহণ, ε-এর মান 0 ধরা। **প্রতি অবশিষ্টের জন্য একটি অবস্থা** $r \in \{0,1,2,3,4\}$ নাও এবং চাল দাও $r \to (2r + b) \bmod 5$: r = 0: 0→0, 1→1 · r = 1: 0→2, 1→3 · r = 2: 0→4, 1→0 · r = 3: 0→1, 1→2 · r = 4: 0→3, 1→4। অবস্থা 0 একই সঙ্গে **শুরু** ও একমাত্র **গ্রহণকারী**।

**ট্রেস 1111** (= 15): 0 -1→ 1 -1→ 3 -1→ 2 -1→ 0 ⇒ গ্রহণ, কারণ $15 \bmod 5 = 0$।

৫টি অবস্থা, এবং এটিই মিনিমাল: 2 mod 5-এর সহজ, তাই 2-এর ঘাতমান দিয়ে সব নন-শূন্য অবশিষ্টে পৌঁছানো যায়, অর্থাৎ যেকোনো দুটি ভিন্ন অবশিষ্টকে কিছুটা 1-এর সম্পূর্ণসূচী আলাদা করে।

# ৮ পাশডাউন অটোমাটন (PDA)

**সংজ্ঞা:** **পাশডাউন অটোমাটন** (pushdown automaton) হলো ফাইনিট অটোমাটনের সঙ্গে একটি **স্ট্যাক** (LIFO মেমরি) — $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$। প্রতি চালে সর্বোচ্চ একটি ইনপুট প্রতীক ও একটি স্ট্যাক-প্রতীক পড়ে স্ট্যাকে ঢোকানো বা বের করা হয়। একটি স্ট্যাক **একটিই সীমাহীন গণনা** মনে রাখতে পারে, তাই PDA গ্রহণ করে ঠিক **কনটেক্সট-ফ্রি** ভাষা।

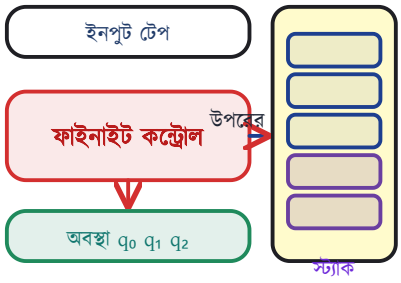
| অংশ | অর্থ |
|---|---|
| **$Q$, $\Sigma$, $q_0$, $F$** | FA ৫-টাপলের হুবহু: অবস্থা, ইনপুট অ্যালফাবেট, শুরু অবস্থা, চূড়ান্ত অবস্থা |
| **$\Gamma$** (নতুন) | স্ট্যাক অ্যালফাবেট: স্ট্যাকে কী কী থাকতে পারে |
| **$Z_0$** (নতুন) | বটম মার্কার, $Z_0 \in \Gamma$; স্ট্যাক $Z_0$ দিয়ে শুরু হয়, শুধু $Z_0$ বাকি থাকলে সেটি "খালি/বেলে" |
| **$\delta$** | $\delta$: $Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \to P(Q \times \Gamma^*)$ — একটি প্রতীক (বা $\varepsilon$) ও স্ট্যাকের উপরের প্রতীক পড়ে, তারপর উপরের প্রতীকটি একটি **স্ট্রিং** দিয়ে বদলে |

## $L = \{a^n b^n \mid n \geq 0\}$, যেখানে $\Sigma = \{a, b\}$ ও $\Gamma = \{A, Z_0\}$

| অবস্থা | ইনপুট | উপরের প্রতীক | উপরেরটির বদলে | কাজ |
|---|---|---|---|---|
| $q_0$ | a | $Z_0$ | A $Z_0$ | প্রথম a-এর জন্য push |
| $q_0$ | a | A | A A | পরের প্রতিটি a-এর জন্য push |
| $q_0 \to q_1$ | b | A | $\varepsilon$ | প্রথম pop — এখন উল্টো গোনা শুরু |
| $q_1$ | b | A | $\varepsilon$ | পরের পর pop করো |
| $q_1$ | $\varepsilon$ | A | $\varepsilon$ | ইনপুট শেষ, স্ট্যাক বেলে $\Rightarrow$ গ্রহণ |
| $q_0 \to q_2$ | $\varepsilon$ | $Z_0$ | $Z_0$ | n = 0: খালি ইনপুটও গ্রহণ |

```
input: a a a b b b
stack: Z0 -> AZ0 -> AAZ0 -> AAAZ0 -> AAZ0 -> AZ0 -> Z0  (q2, accept)
```

**পরীক্ষার ফাঁদ:** উপরের পাঁচটি চাল নিজে থেকে **খালি স্ট্রিং প্রত্যাখ্যান** করে — $q_0$ থেকে বেলা স্ট্যাকে কোনো $\varepsilon$-চাল নেই, অথচ ভাষা লেখা হয় $\{a^n b^n \mid n \geq 0\}$। তাই শেষ সারিতে $\delta(q_0, \varepsilon, Z_0) \to (q_2, Z_0)$ যোগ করতে হয়, অথবা খালি স্ট্যাকে গ্রহণের নিয়ম নিতে হয়। **$n \geq 0$ আর $q_0$ থেকে কোনো $\varepsilon$-চাল নয় — দুটো একসঙ্গে সত্য হতে পারে না।**




ফাইনাইট কন্ট্রোল + একটি স্ট্যাক

**মনে রাখার কৌশল:** a-তে push, b-তে pop, আর গ্রহণ কেবল তখনই যখন স্ট্যাক **বেলে**। "push সমান হতে হবে pop-এর সঙ্গে।"

**প্রচলিত ভুল ধারণা:** **ডিটারমিনিস্টিক** PDA ননডিটারমিনিস্টিক PDA-র সমান নয় — ঠিক উল্টোটা DFA বনাম NFA-র মতো। $\{a^n b^n\}$, $\{w\ c\ w^R\}$ ডিটারমিনিস্টিক কনটেক্সট-ফ্রি; $\{w\ w^R\}$-এর জন্য **NPDA** লাগে, কোনো DPDA পারে না।

## ৯ DFA বনাম NFA বনাম PDA

| বৈশিষ্ট্য | DFA | NFA | PDA |
|---|---|---|---|
| ফর্মাল সংজ্ঞা | ৫-টাপল $(Q, \Sigma, \delta, q_0, F)$ | ৫-টাপল $(Q, \Sigma, \delta, q_0, F)$ | **৭-টাপল** $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ |
| প্রতি ইনপুটে চাল | **ঠিক ১টি** | ০, ১ বা তার বেশি | ০, ১ বা তার বেশি |
| ε-ট্রানজিশন | অনুমোদিত নয় | অনুমোদিত | অনুমোদিত |
| মেমরি | নেই (শুধু অবস্থা) | নেই (শুধু অবস্থা) | **একটি স্ট্যাক** |
| ভাষার শ্রেণি | রেগুলার (টাইপ ৩) | রেগুলার (টাইপ ৩) | কনটেক্সট-ফ্রি (টাইপ ২) |
| ক্ষমতার তুলনা | DFA ও NFA **সমান** — পরস্পরে রূপান্তরযোগ্য | | যেকোনো FA-র চেয়ে কঠোরভাবে শক্তিশালী |
| ডিটারমিনিস্টিক রূপ | সংজ্ঞা অনুযায়ী | ননডিটারমিনিস্টিক | **DPDA < NPDA** — সমান নয় |
| মেশিনের আকার / গতি | সূচকীয়ভাবে বড় হতে পারে, তবে সবচেয়ে দ্রুত | প্রায় ছোট; পিছিয়ে অনুমান লাগে | অবস্থা + স্ট্যাক; ধীর |
| উদাহরণ ভাষা | w 0-এ শেষ হয় | w-তে 01 আছে | $a^n\ b^n$, $w\ w^R$ |

### কোন ভাষার জন্য কোন মেশিন

| ভাষা | সবচেয়ে ছোট মডেল |
|---|---|
| $a^*\ b^*$ · a-এর জোড় সংখ্যক | DFA, 2 states |
| w ab-তে শেষ হয় | **DFA, 3 states** |
| $\{a^n\ b^n \mid n \geq 0\}$ | **DPDA** — ডিটারমিনিস্টিক CF |
| $\{w\ c\ w^R \mid w \in \{a,b\}^*\}$ | **DPDA** — ডিটারমিনিস্টিক CF |
| $\{w\ w^R \mid w \in \{a,b\}^*\}$ | **শুধু NPDA** — কোনো DPDA নয় |
| $\{a^n\ b^n\ c^n\}$ | কোনোটিই নয় — LBA / TM |

### ক্ষমতার সিঁড়ি

ফাইনিট অটোমাটন ⊂ পাশডাউন অটোমাটন ⊂ লিনিয়ার বাউন্ডেড অটোমাটন ⊂ টিউরিং মেশিন। প্রতিটি ধাপে মেমরি বাড়ে এবং **কঠোরভাবে বেশি** ভাষা গ্রহণ হয়: কোনো PDA কনটেক্সট-ফ্রি-বহির্ভূত ভাষা গ্রহণ করে না, কোনো FA $a^n b^n$ গ্রহণ করে না।

**মনে রাখার কৌশল:** "**FA** = **f**ixation (এক অবস্থা), **PDA** = **p**ile (স্ট্যাক), **TM** = **t**ape (অসীম, পড়া ও লেখা যায়)।" মেমরি বাড়ে: স্ট্যাক ⇒ সীমিত টেপ ⇒ টেপ।

**IT link:** টেক্সট এডিটর বন্ধনী মিলিয়ে একটি **স্ট্যাক** ব্যবহার করে — পাতা ৮-এর PDA। *while/if* ঠিকভাবে বাস্তবে নেস্ট করা হয় কি না, সেটা কোনো ফাইনিট অটোমাটনের বৈশিষ্ট্য **নয়**।

**পরীক্ষার ফাঁদ:** যে অসমতা সবাই ভুলে যায়: **DPDA ≠ NPDA**, অথচ **DFA = NFA**। একই পত্রে দুটোই আসে।

# পরীক্ষার আগের রাতের পাতা

**তথ্য-পাতা:**

অ্যালফাবেট $\Sigma$, স্ট্রিং $w$, $|w|$, $\varepsilon$, $\Sigma^*$, $L \subseteq \Sigma^*$

$|\Sigma^n| = |\Sigma|^n \cdot \Sigma^0 = \{\varepsilon\}$

স্টার * = শূন্য বা তার বেশি; প্লাস + = এক বা তার বেশি

$(a+b)^* = \Sigma^*$ ($\varepsilon$-সহ); $(a+b)^+$ $\varepsilon$ বাদ দেয়

$ab^*a$ = a, শূন্য বা তার বেশি b, তারপর a → "aa"-ও

টাইপ ০ ⊃ ১ ⊃ ২ ⊃ ৩ (TM, LBA, PDA, FA)

FA / DFA / NFA = ৫-টাপল $(Q, \Sigma, \delta, q_0, F)$

PDA = ৭-টাপল + $\Gamma$ (স্ট্যাক অ্যালফাবেট) + $Z_0$ (বটম)

সম্পূর্ণ DFA: প্রতি (অবস্থা, প্রতীক)-এ ঠিক ১টি চাল

NFA: ০ বা তার বেশি চাল; $\varepsilon$-চাল অনুমোদিত

DFA ≡ NFA; সাবসেট → সর্বোচ্চ $2^n$ অবস্থা, সীমা নিবিদ্

DPDA < NPDA — ব্যতিক্রম, নিয়ম নয়

"ab-তে শেষ হয়" মিনিমাল DFA = 3 states

বাইনারি mod 5 (MSB আগে) = 5 states, $r \to (2r+b) \bmod 5$

পাম্পিং লেমা প্রমাণ করে ভাষা **রেগুলার নয়**, কখনোই রেগুলার

$\varepsilon \in \{a^n b^n \mid n \geq 0\}$: PDA খালি স্ট্রিং গ্রহণ করে

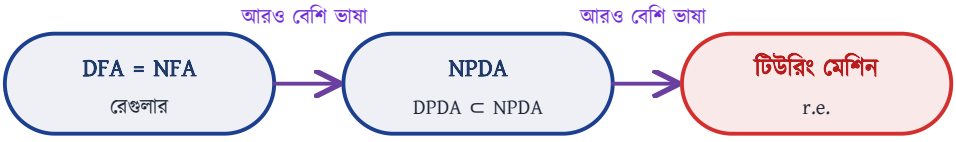
## MCQ-র জন্য সেরা ১২টি এক লাইনের তথ্য

1. DFA = ৫-টাপল, PDA = ৭-টাপল
2. সম্পূর্ণ DFA ⇒ ঠিক একটি ট্রানজিশন
3. DFA ও NFA একই ভাষা গ্রহণ করে
4. সাবসেট কনস্ট্রাকশন ≤ $2^n$ DFA অবস্থা
5. PDA ⇒ কনটেক্সট-ফ্রি, এর বেশি নয়
6. স্টার শূন্য রাখতে পারে, প্লাস পারে না
7. $\{a^n b^n\}$ **রেগুলার নয়** — PDA লাগে
8. "ab-তে শেষ হয়"-এ 3 অবস্থা লাগে
9. ৫-ভাগাতীয় বাইনারির জন্য 5 অবস্থা
10. পাম্পিং লেমা কেবল "রেগুলার নয়" প্রমাণ করে
11. $\{w\ w^R\}$-এর জন্য ননডিটারমিনিস্টিক লাগে
12. $\varepsilon \in \{a^n b^n \mid n \geq 0\}$

**প্রশ্নটা কোন মেশিন চাইছে?:** **অবস্থা, মেমরি নেই, ৫-টাপল** উল্লেখ ⇒ DFA · "অনুমান", **একাধিক চাল** বা **ε** ⇒ NFA · **স্ট্যাক, ৭-টাপল**, $a^n b^n$-ধরনের ভাষা ⇒ PDA · **দুটি স্ট্যাক / টেপ** লাগে বা কনটেক্সট-ফ্রিই নয় ⇒ TM।

**সবচেয়ে বড় ফাঁদ:** "শূন্য বা তার বেশি ট্রানজিশন" NFA-র বৈশিষ্ট্য · DFA-র টেবিলে ফাঁকা ঘর মানে অসম্পূর্ণ মেশিন — ট্র্যাপ অবস্থা যোগ করো · পাম্পিং লেমা মেনে চললেই রেগুলার প্রমাণ হয় না · সাবসেট-কনস্ট্রাকশনের DFA মিনিমাল নাও হতে পারে · DFA = NFA কিন্তু DPDA ≠ NPDA · $n \geq 0$ মানে খালি স্ট্রিংও ধরা

**সব কৌশল একসঙ্গে:** "স্টার = শূন্য বা বেশি, প্লাস = এক বা বেশি" · "পাঁচটি অক্ষর, তারপর $\Gamma$ আর $Z_0$" · "DFA জানতে হবে, NFA অনুমান করতে পারে" · "push সমান হতে হবে pop-এর সঙ্গে" · "FA = fixation, PDA = pile, TM = tape" · "একটি স্ট্যাক একটি ভারসাম্য গোনে"




প্রতিধাপ বাড়ে = আরও এক ধরনের মেমরি

প্রথম ধাপে সমান, এরপর প্রতিটি ধাপে কঠোর বৃদ্ধি

## ✓ অনুশীলন MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

**১.** DFA কতগুলো অংশের টাপল?

ক) 3 খ) 4
গ) 5 ঘ) 7

**২.** **সম্পূর্ণ** DFA-তে প্রতি অবস্থার প্রতি প্রতীকে:

ক) ঠিক 1 move খ) সর্বোচ্চ 1
গ) 0 বা বেশি ঘ) অন্তত 2

**৩.** একটি NPDA ঠিক কোন ভাষা গ্রহণ করে?

ক) রেগুলার খ) কনটেক্সট-সেনসিটিভ
গ) রি. এনিউমারেবল ঘ) কনটেক্সট-ফ্রি

**৪.** {a, b}-এর উপর (a + b)* কী বোঝায়?

ক) সব স্ট্রিং, ε-সহ খ) খালি-নয় স্ট্রিং
গ) #a = #b ঘ) শুধু $a^n b^n$

**৫.** কোনটি **রেগুলার নয়**?

ক) a* b* খ) (ab)*
গ) a-এর জোড় সংখ্যা ঘ) $\{a^n\ b^n \mid n \geq 0\}$

**৬.** ab*a-এর ভাষায় কোন স্ট্রিং আছে?

ক) ab খ) aa
গ) ba ঘ) abab

**৭.** n-অবস্থার NFA থেকে সর্বোচ্চ কত অবস্থা?

ক) n খ) 2n
গ) $n^2$ ঘ) $2^n$

**৮.** "ab-তে শেষ হয়"-এর মিনিমাল DFA:

ক) 2 states খ) 3 states
গ) 4 states ঘ) 5 states

**৯.** পাম্পিং লেমা প্রমাণ করে ভাষা:

ক) মিনিমাল খ) রূপান্তরযোগ্য
গ) রেগুলার নয় ঘ) রেগুলার

**১০.** ৫-ভাগাতীয় বাইনারির মিনিমাল DFA:

ক) 5 states খ) 2 states
গ) 4 states ঘ) 32 states

উত্তরমালা → ১-গ · ২-ক · ৩-ঘ · ৪-ক · ৫-ঘ · ৬-খ · ৭-ঘ · ৮-খ · ৯-গ · ১০-ক

**লিখিত প্রশ্ন:** {0, 1} অ্যালফাবেটের উপর এমন DFA বানাও যা ঠিক **01**-এ শেষ হওয়া স্ট্রিংগুলো গ্রহণ করে। ট্রানজিশন টেবিল দাও এবং *1101* ইনপুটে ট্রেস করো। ৮ নম্বর

**মডেল উত্তর ✍️:**

১ **অবস্থা** (01-এর দিকে অগ্রগতি): $q_0$ শুরু, কিছু মেলেনি; $q_1$ শেষ প্রতীক ছিল 0; $q_2$ শেষের দুটি প্রতীক 01 ছিল — **গ্রহণকারী** 2

২ **টেবিল**, প্রতি প্রতীকে একটি চাল:

| অবস্থা | 0 পড়লে | 1 পড়লে |
|---|---|---|
| → $q_0$ | $q_1$ | $q_0$ |
| $q_1$ | $q_1$ | $q_2$ |
| * $q_2$ | $q_1$ | $q_0$ |

$M = (\{q_0, q_1, q_2\}, \{0,1\}, \delta, q_0, \{q_2\})$ 3

৩ **ট্রেস 1101:** $q_0$ -1→ $q_0$ -1→ $q_0$ -0→ $q_1$ -1→ $q_2 \in F \Rightarrow$ **গৃহীত**; মিনিমাল, কারণ ইনপুট 1-এ $q_0$ ও $q_1$ আলাদা হয় 3

**মুখস্থের ছক:** k-প্রতীকের কোনো সাফিক্স মানে সবসময় **k+1** অবস্থা লাগে: $q_0$ কিছু মেলেনি ... $q_k$ পুরো সাফিক্স এইমাত্র পড়া হয়েছে (গ্রহণকারী)।

## পরীক্ষার আগে যাচাই ✓

- ✓ দুটো টাপলই মেমরি থেকে লিখতে পারি
- ✓ "ends with" ও "contains"-এর DFA-NFA আঁকতে পারি
- ✓ 3-অবস্থার NFA-তে সাবসেট কনস্ট্রাকশন চালাতে পারি

**লিখিত অনুশীলন:**

① টেবিল-ফিলিং করে 5-অবস্থার DFA মিনিমাইজ করো

② "w-তে ab আছে" ε-NFA-কে DFA-তে রূপান্তর করো